DE MONTFORT UNIVERSITY CAMBODIA

Operating Systems & Networks
CTEC1704C

Lecture – 5 : CPU Architecture

Dr. Sameer Jha, SFHEA
Faculty of Computing
De Montfort University Cam o!ia

DE MONTFORT UNIVERSITY CAMBODIA

Outline

$ CPU Architecture and Function

$ What is a Register?

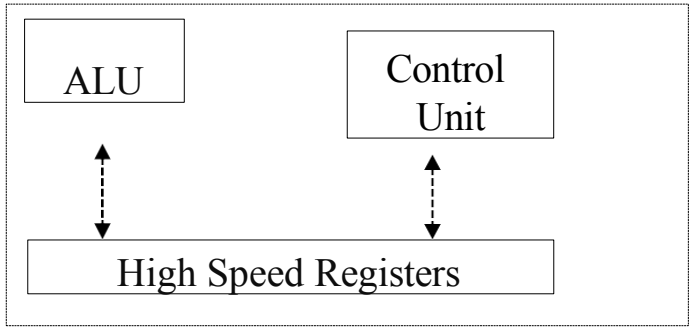
$ CPU functions

$ CPU performance

$ Pipelining

Central Processing Unit Architecture

- The Central Processing Unit is composed of three main components:
  - Control Unit - to fetch & decode instructions
  - ALU to perform arithmetic(*/+-) and logical operations (AND,OR,NOT)
  - Registers to store temporary data.



CPU

The number and design of registers are also crucial to CPU architecture

Control Unit

&he control unit !eci!es *what happens next* in the C+U.

Example to mention

- ,u-er over.o(attac's re!irect the control unit to malicious instructions

Endpoint security relevance

- Mal(are alters e/ecution .o(using control logic

$E/ploits hi0ac' control .o(1e.g. return2oriente! programming3

$Control2.o(integrity 1CF)3 is a mo!ern !efence

ALU (Arithmetic Logic Unit)

A4U 1Arithmetic 4ogic Unit3 is responsi le for performing arithmetic an! logical functions or operations.)t consists of t(o su sections, (hich are5

- Arithmetic Section5 ,y arithmetic operations, (e mean operations li'e a!!ition, su traction, multiplication, an! !ivision, an! all these operations an! functions are performe! y A4U.
- 4ogic Section5 ,y 4ogical operations, (e mean operations or
  functions li'e selecting, comparing, matching, an! merging the !ata, an! all these are performe! y A4U.

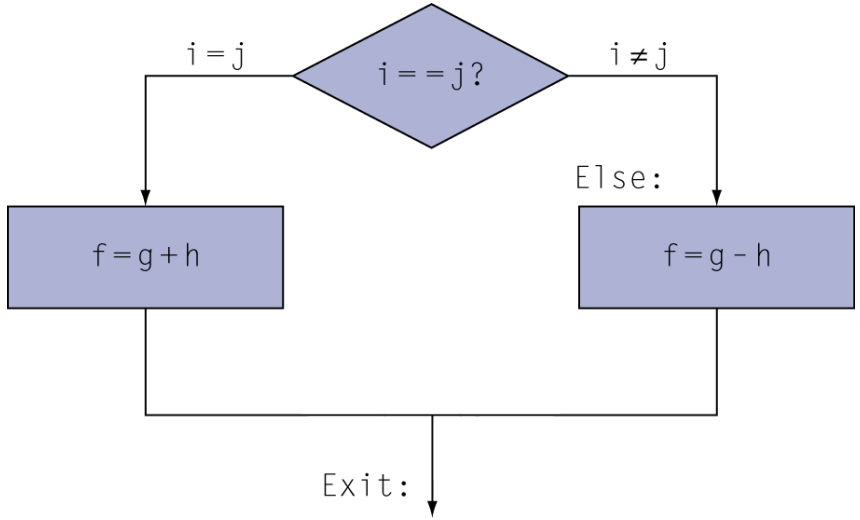
6ote5 &he C+U may contain more than one A4U an! it can e use! for maintaining timers that help run the computer system.

Conditional Operations

- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
-
  - if (rs == rt) branch to instruction labeled L1
-
  - if (rs != rt) branch to instruction labeled L1
-
  - unconditional jump to instruction labeled L1

Compiling If Statements

- C code:

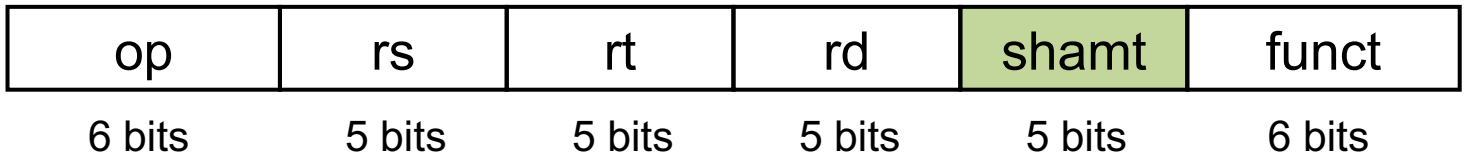


- f, g, … in $s0, $s1, …
- Compiled MIPS code:

```
# go to ‘Else’ if i!=j
# f=g+h (skip if i!=j)
# go to ‘Exit’
        # f=g-h (skip if i==j)
```

Assembler calculates addresses

Shift Operations



- shamt: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - sll by i bits **multiplies** by 2^i
- Shift right logical
  - Shift right and fill with 0 bits
  - srl by i bits **divides** by 2^i (unsigned only)

CPU Function

What Does a CPU Do?

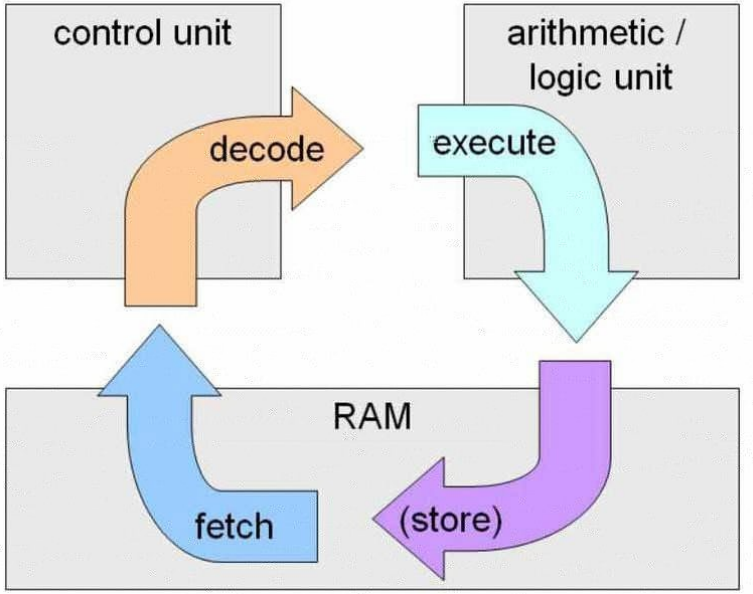
&he main function of a computer processor is to e/ecute instructions an! pro!uce an output.

Fetch, Decode, an! E/ecute are the fun!amental functions of the computer.

$ **Fetch:** the 9rst C+U gets the instruction. &hat means inary num ers that are passe! from :AM to C+U.

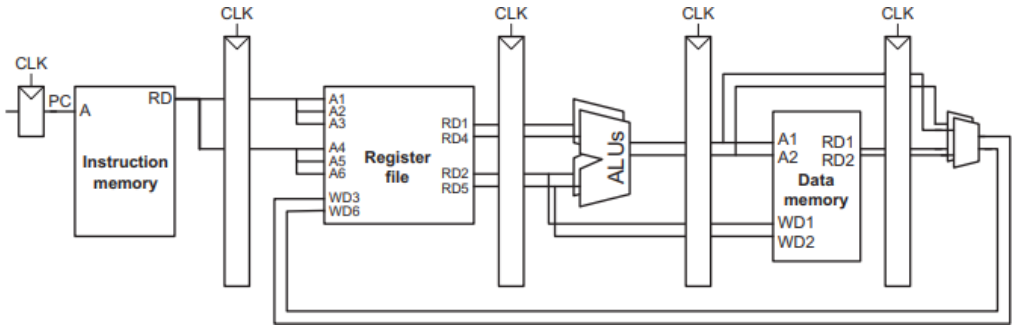
$ **Decode:** ; hen the instruction is entere! into the C+U, it nee!s to !eco!e the instructions. (ith the help of A4U1Arithmetic 4ogic Unit3, the process of !eco!ing egins.

$ **Store:** After the e/ecute step the instructions are ready to store in the



CPU Instruction Cycle

- Programs consist of many instructions
- Processor follows an **instruction cycle** to execute each instruction
- Which steps do we need in an instruction cycle?
  - IF: Instruction fetch from memory
  - ID: Instruction decode & register read
  - EX: Execute operation or calculate address
  - MEM: Access memory operand
  - WB: Write result back to register



This is a core security concept :
Malware and security software both rely on the same instruction cycle.

Instruction Execution

IF

- Load PC into instruction memory, fetch instruction

ID

- Look up register numbers in register file, read registers

EX

- **Depending on instruction class**
  - Use ALU to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch target address

MEM

  - Access data memory for load/store
  - For next cycle: PC = target address or PC + 4

CPU performance

Even though today's processors are tremendously fast, their performance can be affected by a number of factors:

- Clock speed
- Cache memory size
- Number of cores

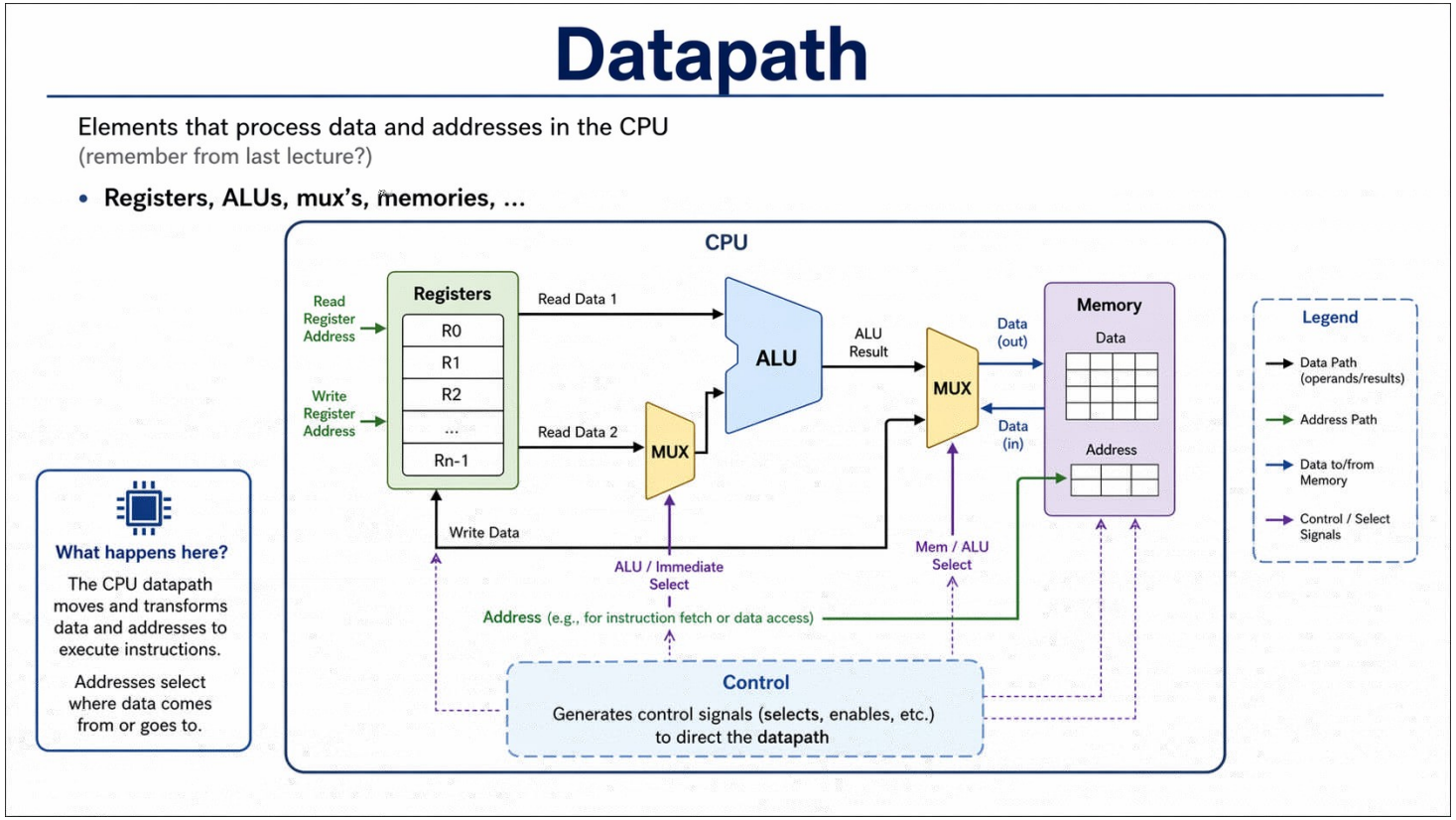
eal!world example

•Cryptomining mal (are pushes C+U cloc ' usage to ma/imum

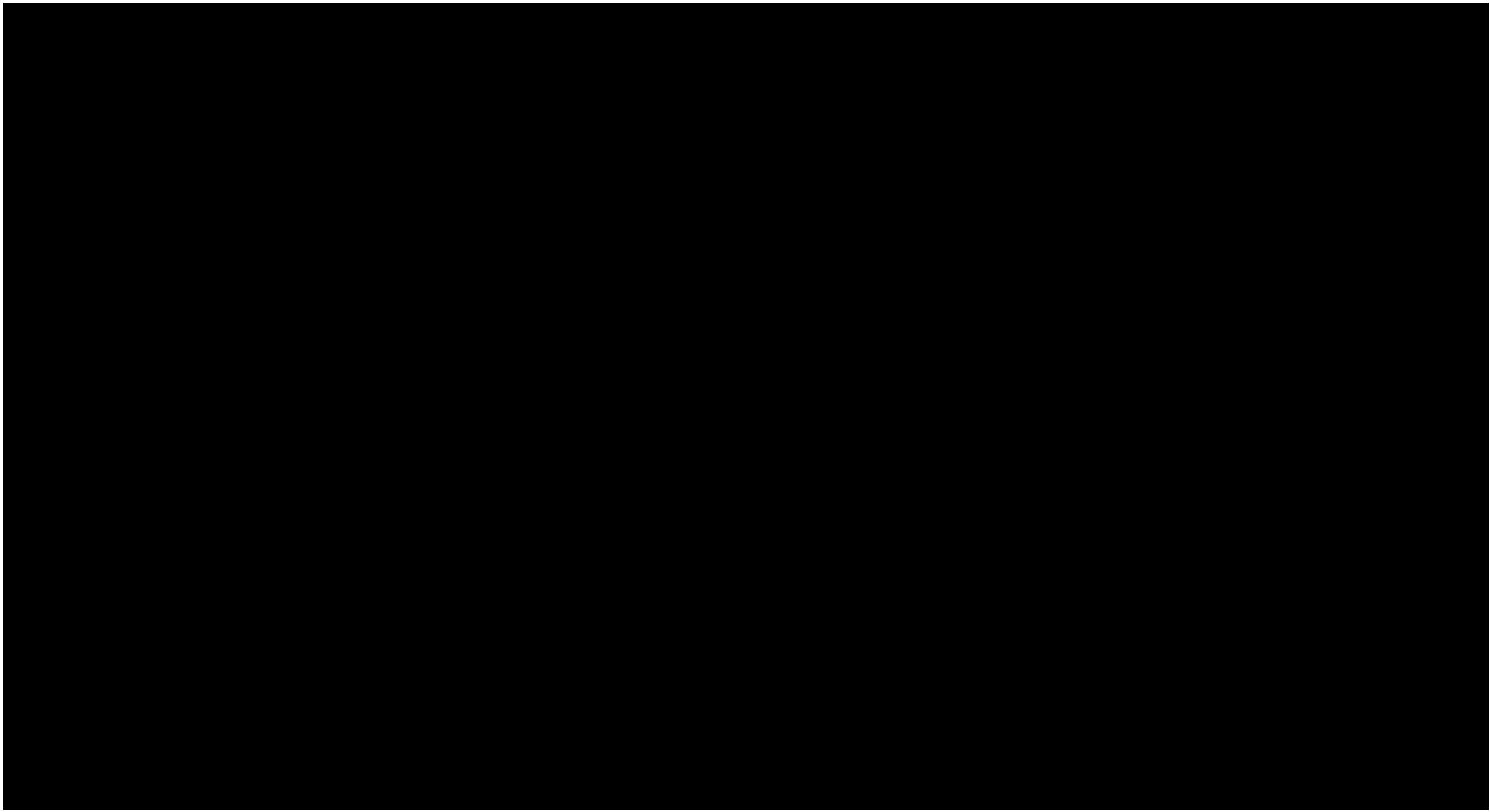
$En ! point tools . ag sustained high " #U usage

Building a Datapath

- Datapath
  - Elements that process data and addresses in the CPU
    - Registers, ALUs, mux's, memories, …

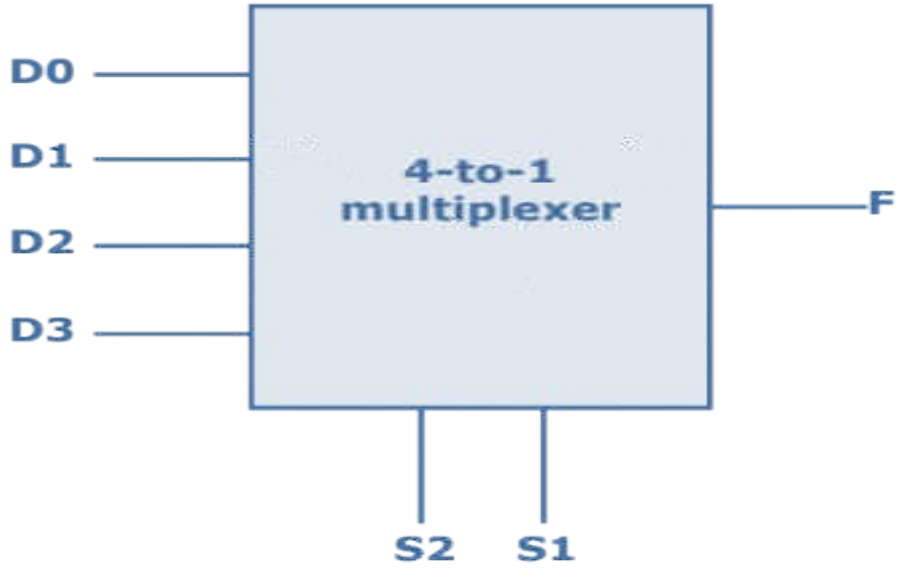


Useful Combinational Circuits



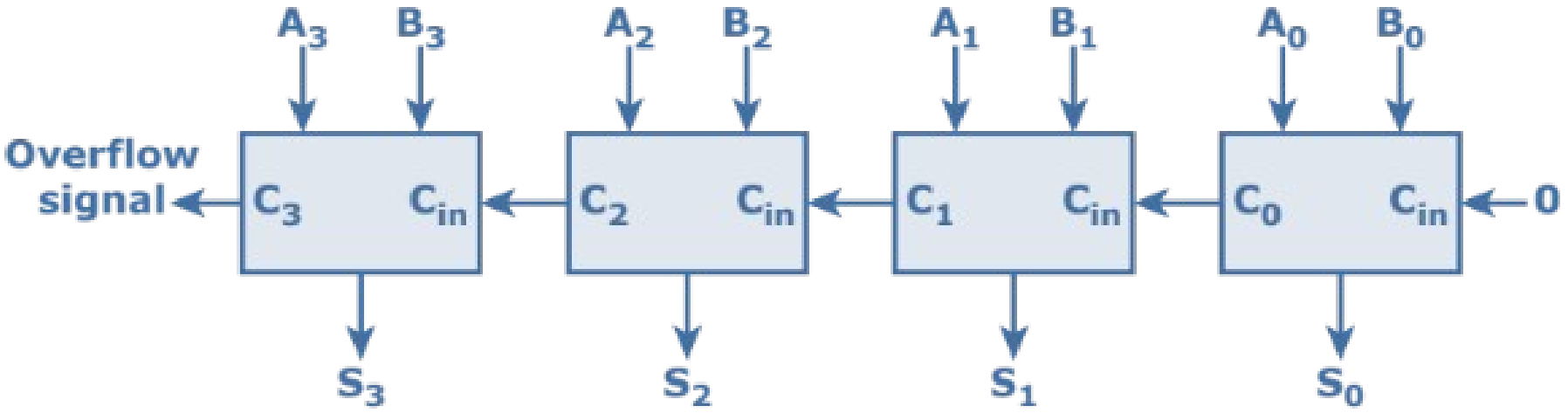
Multiplexer

- Used to select one from several inputs (D)
- Selection code (S) specifies which input



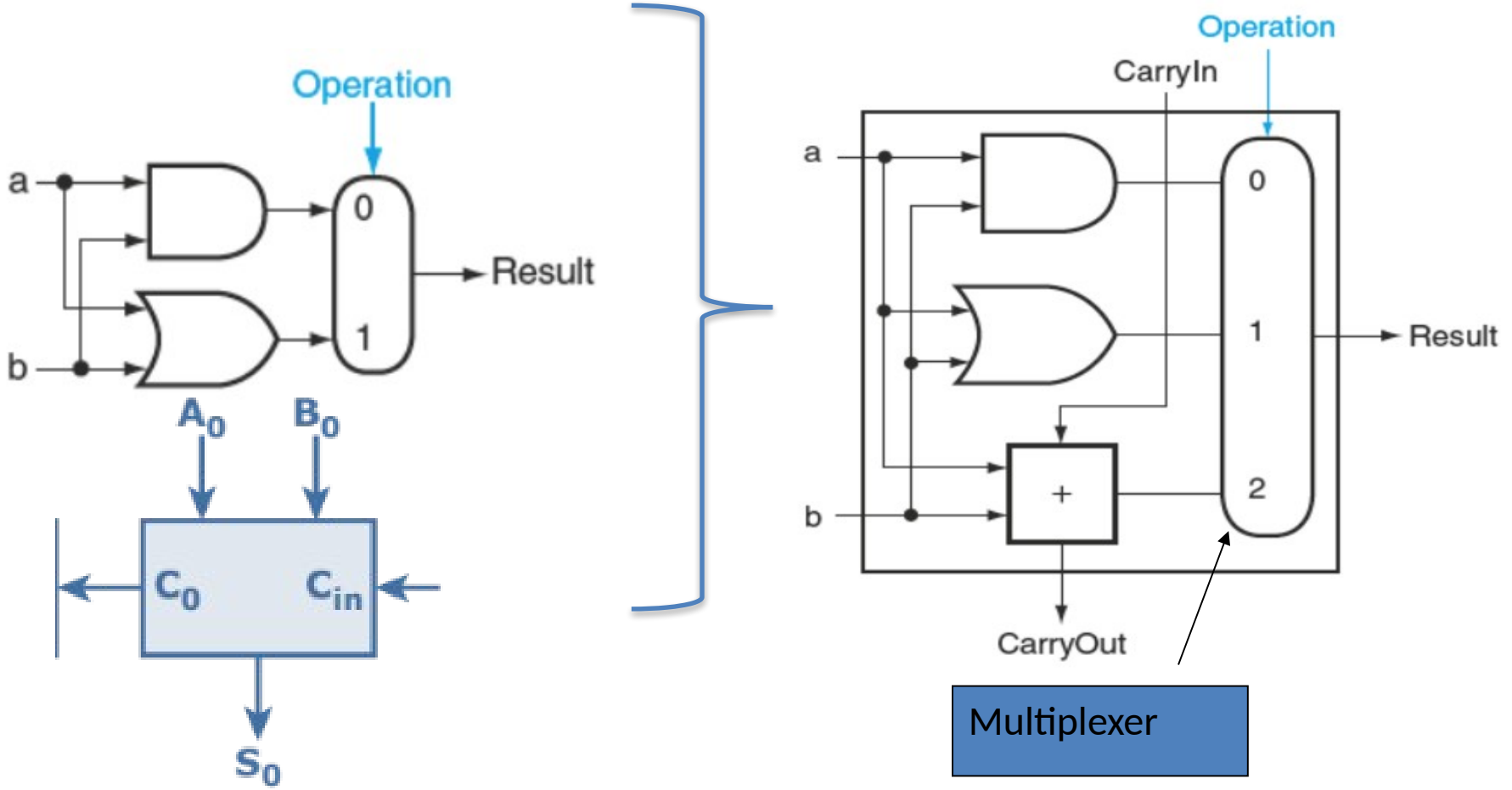
4-bit Adder

We can combine single-bit adders by chaining the carry bits: connect carry-out of adder 0 to carry-in of adder 1



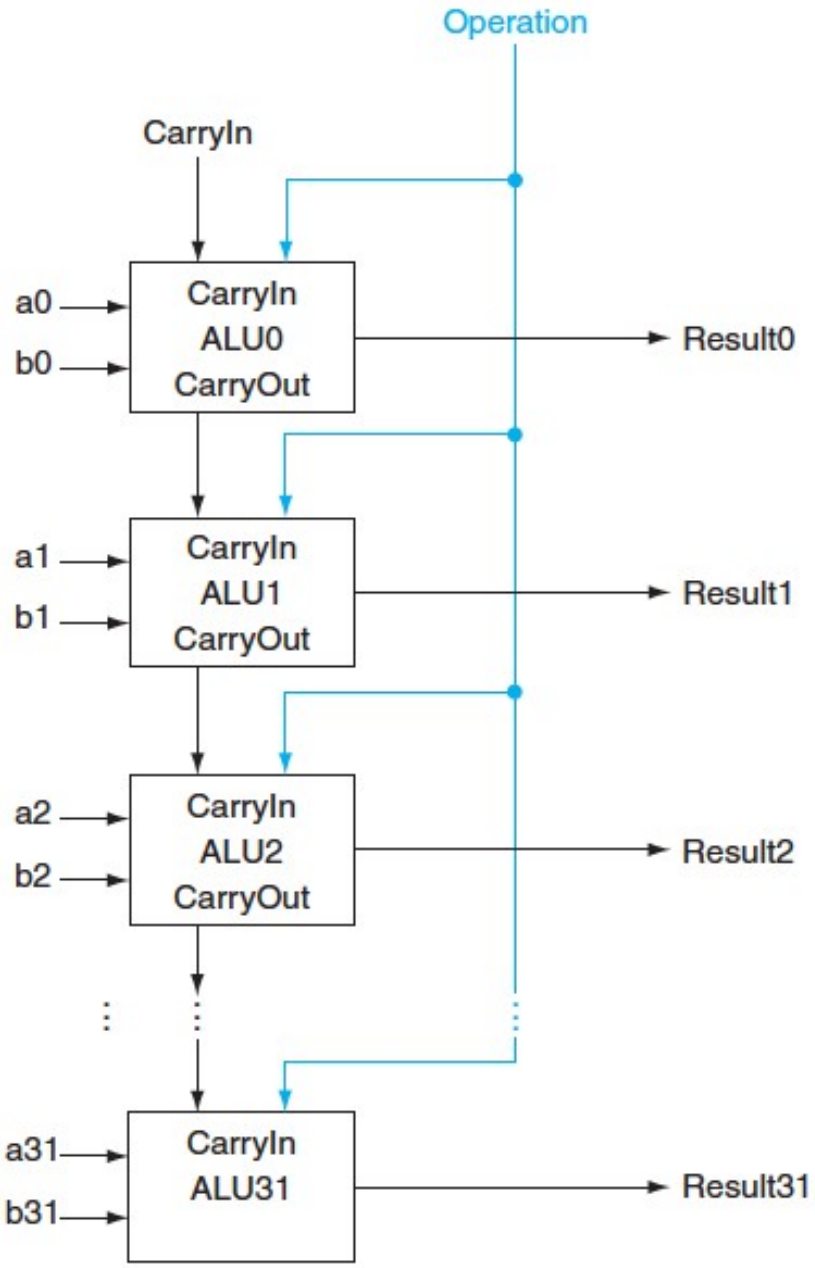
1-bit Arithmetic-Logic Unit (ALU)

Combine 1-bit AND/OR unit with 1-bit adder



32-bit ALU

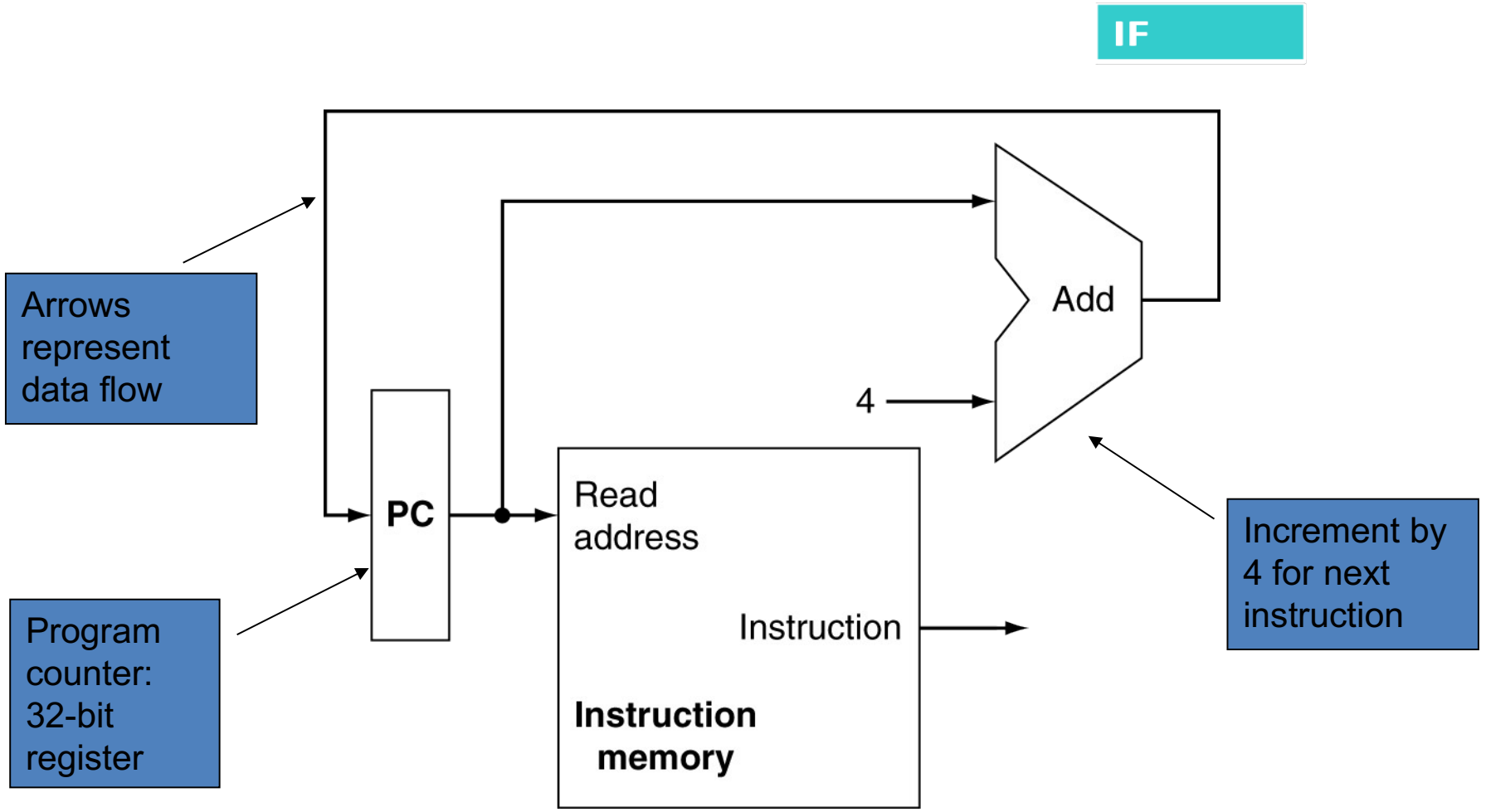
- Chain 32 1-bit ALUs using the carry bits
- The result bits from the 32 1-bit ALUs form the 32-bit result



Moving on….

How do they come together in a DataPath??

Start with Instruction Fetch

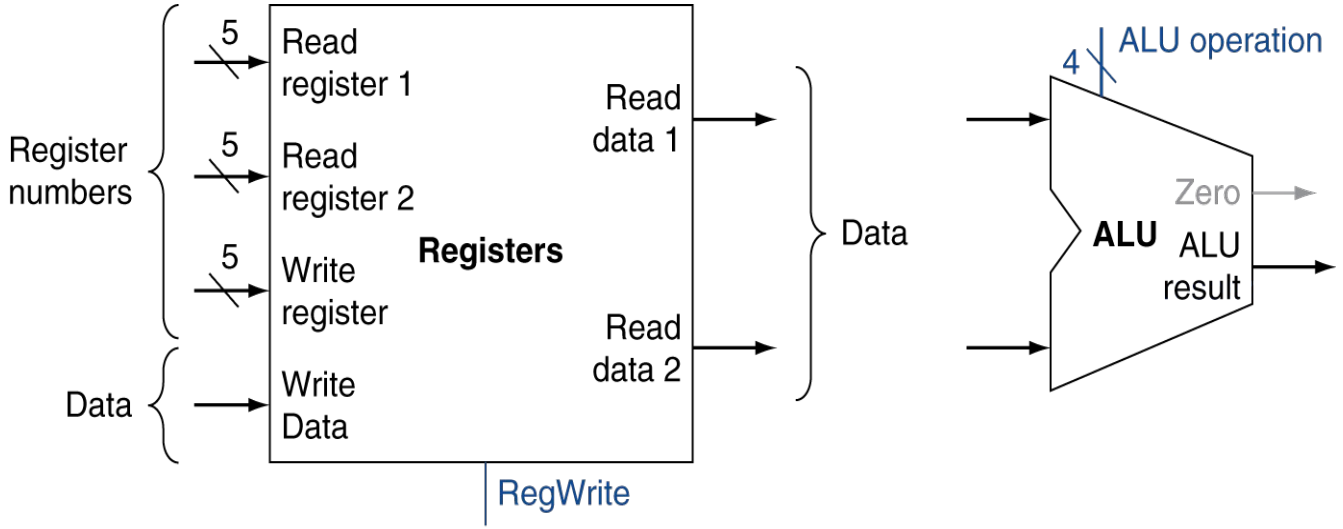


Arithmetic/Logical Instructions

ID
- Read two register operands

EX
- Perform arithmetic/logical operation

WB
- Write register result



a. Registers

b. ALU

Load/Store Instructions

ID

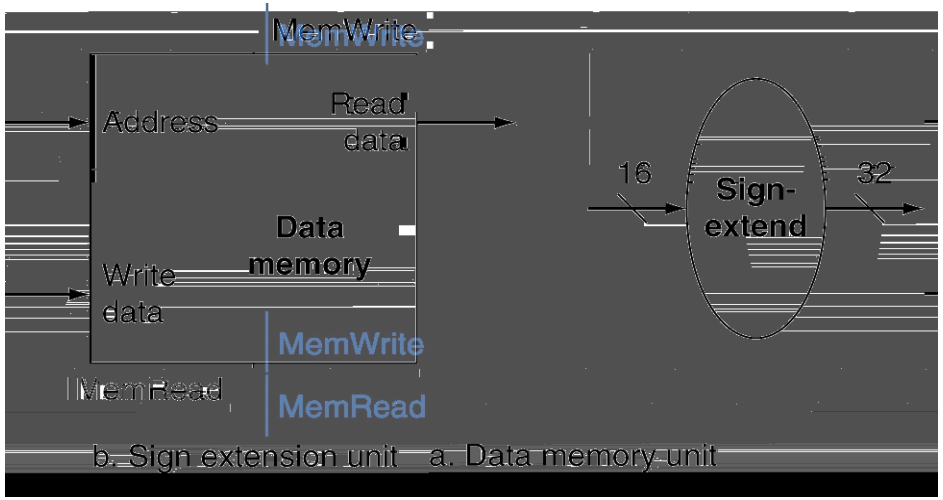
- Read register operands

EX

- Calculate address using 16-bit address
  - Use ALU, but sign-extend address (needed because instruction supplies 16 bits, but ALU expects 32)

MEM

- Load: Read memory and update register
- Store: Write register value to memory



Branch Instructions

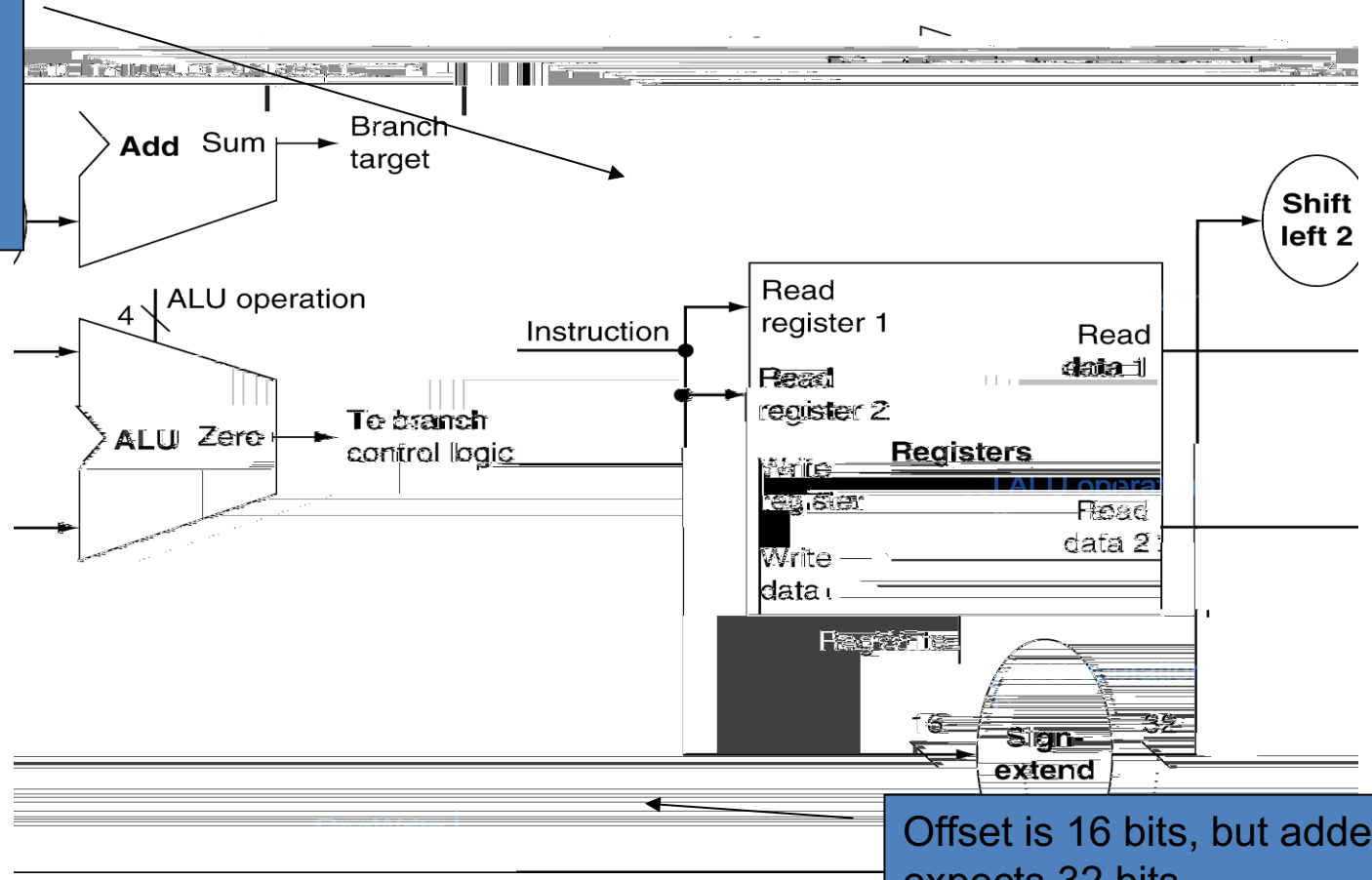
ID
- Read register operands

EX
- Compare operands
  - Use ALU, subtract and check Zero output

EX
- Calculate target address
  - Sign-extend 16-bit address (label)
  - Shift left 2 places (address is given in **words**, i.e. multiples of 4 bytes. We shift left to multiply with 4 and get the target address in bytes)
  - Add to PC + 4
    - This is done by the instruction fetch stage

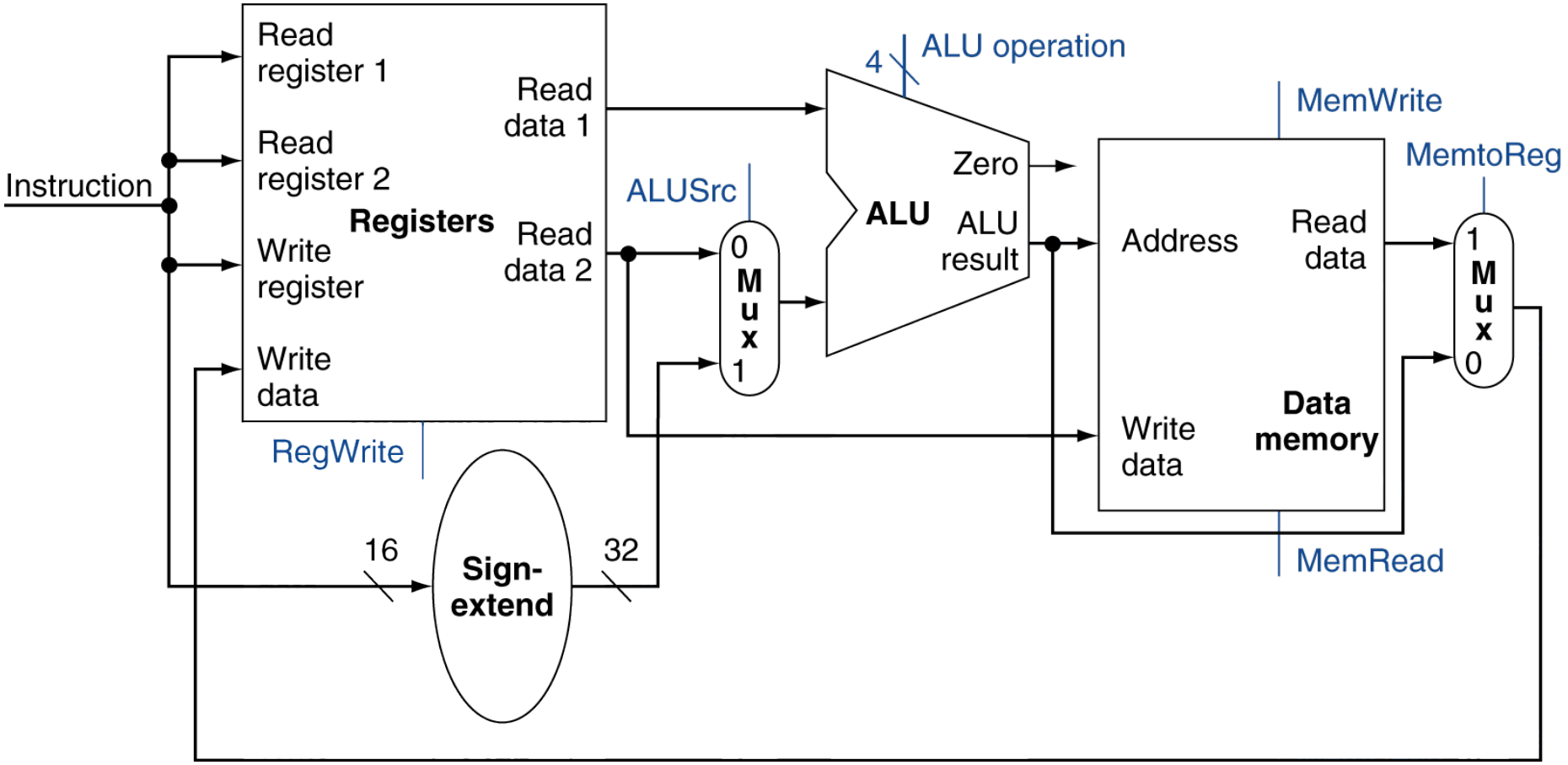
Branch Instructions

Multiplies with 4 to get the target address in bytes

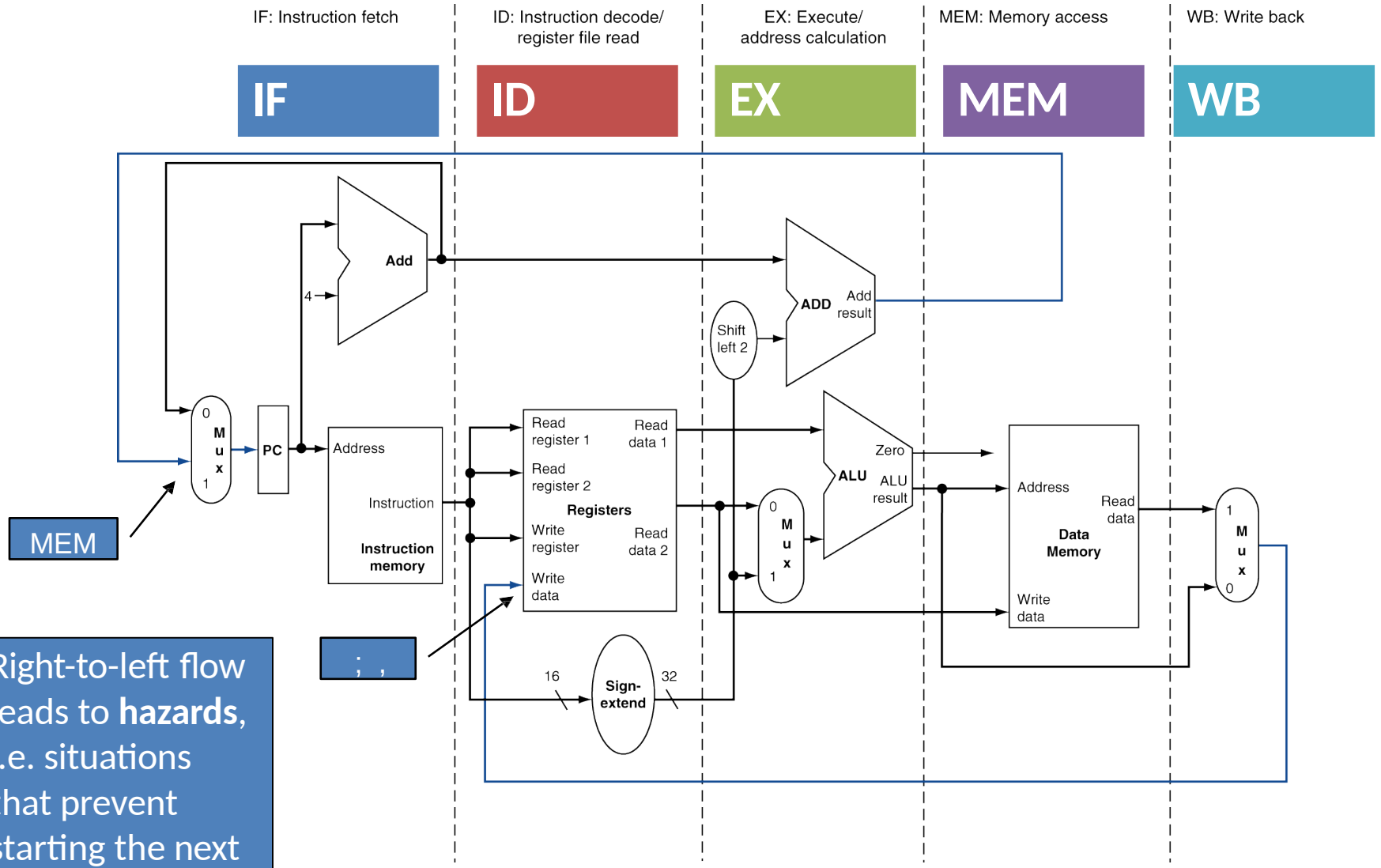


Offset is 16 bits, but adder expects 32 bits

Arithmetic/Logical/Load/Store Datapath



MIPS Pipelined Datapath

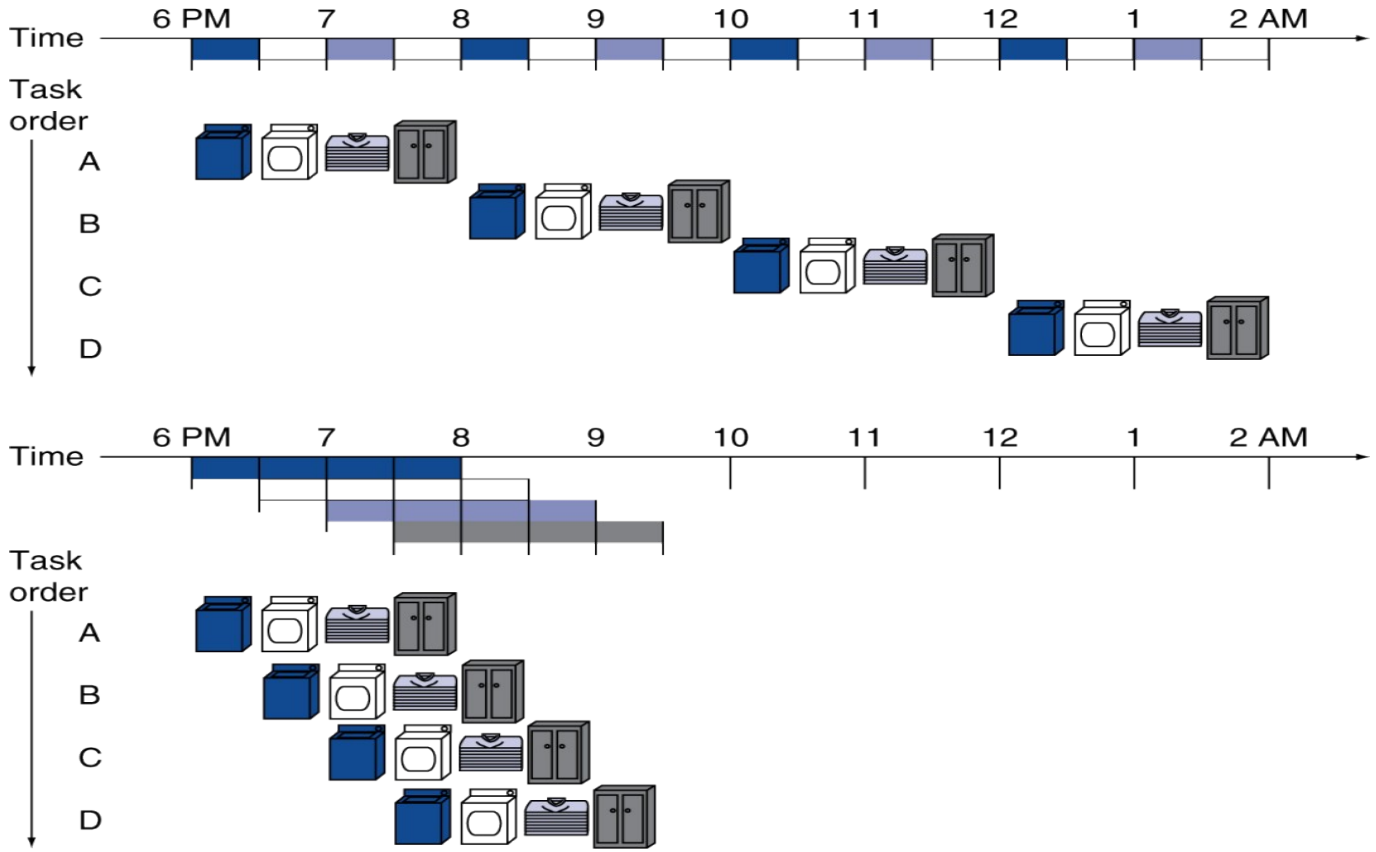


Right-to-left flow leads to **hazards**, i.e. situations that prevent starting the next instruction

Pipelining Analogy

$ Pipelined laundry: overlapping execution

< Parallelism improves performance



Summary

- You see, everything is MATHS.
- At the EOD, Mathematical operations and logical diagrams model the world.
- These models are run on computers with the help of hardware components, building up on layers and layers of abstractions .
- But at the very base of it, it’s basic maths and logic, and we learnt how it works.

DE MONTFORT UNIVERSITY CAMBODIA

DE MONTFORT UNIVERSITY CAMBODIA